

Lab 2 – Mod3 – Contenerización

Luis Jesús Ferrufino Burgos

Se uso el siguiente contenedor para las pruebas:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
47e19d30b8d8	nginx:alpine	"/docker-entrypoint..."	41 seconds ago	Up 40 seconds
0.0.0.0:8080->80/tcp	[::]:8080->80/tcp	testcontainer		

Preguntas

1. PID del contenedor en el host:

"Pid": 401

```
luis@DESKTOP-49D4VBP:~$ docker inspect --format '{{.State.Pid}}' testcontainer
401
```

2. Cadena procesos padre (pstree)

Investigué un comando similar para cmd

```
C:\Users\HP>wmic process where "name like '%docker%'" get Name, ProcessId, ParentProcessId
Name                                ParentProcessId  ProcessId
Docker Desktop.exe                 10412            4144
com.docker.backend.exe              4144             21172
com.docker.backend.exe              21172            19712
Docker Desktop.exe                 19712            18436
com.docker.build.exe                19712            8924
Docker Desktop.exe                 18436            17792
Docker Desktop.exe                 18436            3024
Docker Desktop.exe                 18436            21396
Docker Desktop.exe                 18436            22492
```

3. Namespace que pertenece el proceso

Los namespaces del proceso 401 aíslan por completo sus recursos en el kernel. Los principales a los que pertenece son:

- **net** (Network): Mapea el entorno de red aislado donde el contenedor tiene asignada la IP interna 172.17.0.2 sobre la interfaz virtual bridge.
- **pid** (Process ID): Permite que el proceso de Nginx se ejecute como el PID 1 dentro del contenedor, permaneciendo completamente ciego al PID 401 real del host.
- **mnt** (Mount): Aísla los puntos de montaje del sistema de archivos, permitiendo que el contenedor use sus capas a través del driver overlayfs de forma independiente.
- **ipc**, **uts**, **user** y **cgroup**, completando el aislamiento total de comunicación, nombres de host y usuarios.

4. Cgroup del proceso

El proceso con el PID 401 está asignado a un grupo de control jerárquico dentro del motor de Docker Desktop. Al revisar la configuración de recursos en el bloque "HostConfig", los campos "Memory": 0, "CpuShares": 0 y "NanoCpus": 0 confirman que el contenedor no tiene límites de hardware establecidos. Por lo tanto, el proceso puede consumir dinámicamente los recursos disponibles que WSL 2 le asigne al entorno global de Docker.

5. ¿Qué pasa si matas el containerd mientras el container corre?

El contenedor testcontainer seguirá corriendo sin interrupción. Esto se debe a que la ejecución del proceso depende directamente de **containerd-shim-v2** (el cual maneja los flujos de entrada/salida y mantiene el contenedor aislado). Si el servicio de **containerd** se detiene o se mata de forma abrupta, el **shim** mantendrá vivo el proceso de Nginx. Al reiniciarse **containerd**, el motor leerá el estado actual y volverá a tomar el control del contenedor de manera transparente.

6. Que flags aplican namespaces / cgroups / capabilities?

Al ejecutar docker run, los parámetros modifican estos tres pilares del kernel:

- **Namespaces:** "NetworkMode": "bridge".
- **Cgroups:** Al inspeccionar muestra "Memory": 0 y "CpuShares": 0 (sin límites). Para aplicar restricciones a través de cgroups, se usan los flags -m 512m (límite de RAM) o --cpus="2" (límite de núcleos).
- **Capabilities:** Se figuran "CapAdd": null y "CapDrop": null. Para añadir o remover permisos de superusuario a nivel de kernel, se pasan los flags --cap-add o --cap-drop.

7. Diferencia docker stop vs docker kill

docker stop testcontainer: Envía un "StopSignal": "SIGQUIT". Nginx intercepta esta señal y cierra de manera limpia y ordenada todas las conexiones HTTP activas de los usuarios antes de finalizar.

docker kill testcontainer: Envía un "SIGKILL" directo al kernel sobre el PID 393. Detiene la ejecución en un instante, interrumpiendo abruptamente cualquier transferencia de datos en el puerto 8080.

8. ¿Por qué docker rm no funciona en un container corriendo?

En este estado, el motor de almacenamiento "Driver": "overlayfs" mantiene los descriptores de archivos del disco abiertos en caliente y la interfaz de red tiene recursos del host asignados. Intentar borrar el contenedor mientras escribe en disco y procesa red rompería la consistencia del motor de Docker. Para eliminarlo, primero se detiene sus procesos (**docker stop**) o forzar la remoción con docker rm -f.